

中山大学

《软件工程》期末大作业

架构设计文档

项目名称：逸仙活动云——中山大学校园活动云平台

小组成员：钟梓俊 邱剑波 张宸睿

指导老师：王可泽

2026 年 7 月 16 日

目录

1	架构设计概述	1
1.1	设计目标	1
1.2	设计原则	1
1.3	设计约束	1
2	系统总体架构	2
2.1	架构模式	2
2.2	系统分层	2
2.3	各层职责	3
3	功能模块设计	3
3.1	模块划分	3
3.2	模块职责	4
3.3	模块依赖关系	4
4	数据与接口设计	5
4.1	数据存储方案	5
4.2	核心数据关系	5
4.3	模块接口	5
4.4	外部接口	6
5	关键设计决策	6
5.1	技术选型	6
5.2	关键方案说明	7
5.3	方案权衡	8
6	架构质量属性	8
6.1	可维护性	8
6.2	可扩展性	8
6.3	性能与可靠性	9

1 架构设计概述

1.1 设计目标

系统架构围绕四个核心目标展开：

1. **可演示**：支持在本地开发环境（SQLite + Mock）完整运行核心业务流程，不依赖外部邮件、搜索或 AI 服务即可展示活动浏览、认证、发布审核和个人参与等主要功能。
2. **可联调**：前后端通过预定义的 REST/JSON 接口契约并行开发，Mock 服务允许前端在无后端环境时独立验证界面逻辑。
3. **可部署**：提供 Docker Compose 容器化部署拓扑，单命令即可在 Linux 服务器上重建完整运行环境。
4. **可演化**：分层架构和服务层隔离确保业务规则变更时影响范围可控，领域名称适配层隔离前后端数据结构差异。

1.2 设计原则

1. **接口优先**：前后端以接口契约为核心协作依据，字段、枚举、分页和时区等约定在编码前达成一致。
2. **接口隔离**：前端通过 `api/` 模块封装所有 HTTP 请求，页面组件不直接调用 Axios 或处理 JSON 序列化，接口变更时只需修改对应的 API 封装层。后端服务层与接口层通过服务函数签名解耦。
3. **最小权限**：每个 API 端点服务端校验 JWT 身份与 RBAC 角色，前端不单独承担授权职责。
4. **可观测性**：关键操作写入审计日志，搜索日志脱敏记录，异常统一格式化为 JSON 响应。
5. **配置外置**：密钥、数据库连接、外部服务地址通过环境变量注入，不硬编码进入源码。

1.3 设计约束

1. **技术栈**：前端使用 Vue 3 + TypeScript + Vite + Pinia，后端使用 Flask + SQLAlchemy + JWT，数据库使用 PostgreSQL（开发环境可用 SQLite 替代）。
2. **部署环境**：课程项目以单机可演示为首要目标，Docker Compose 定义完整服务拓扑；生产环境需额外补充反向代理、TLS 和监控。
3. **外部依赖**：邮件发送、外部搜索和 AI 增强为可选能力，设计降级方案确保核心流程不因外部不可用而中断。

4. **开发周期：**项目周期约 12 周，前端与后端并行开发，通过 Mock 服务和接口契约解耦联调依赖。

2 系统总体架构

2.1 架构模式

前端开发阶段中，Vite 将 /api 请求代理到后端开发服务器；生产环境则由反向代理统一接管静态资源分发、API 路由和 TLS 终止。API 服务将耗时任务（抓取、AI 处理、定时通知）封装为消息放入 Redis 队列，Celery Worker 异步消费，避免阻塞 Web 请求。

系统采用三种核心架构模式：

1. **前后端分离模式：**前端为 Vue 3 单页应用，后端为 Flask REST API，通过 HTTP JSON 通信。两者独立构建、独立部署，通过接口契约保证一致性。前端在开发期可使用 Mock 服务先行开发，不等待后端接口就绪。
2. **分层架构模式：**后端按职责划分为接口层、服务层、持久层和基础设施层，每层只与相邻层交互。服务层封装可复用的业务规则，不依赖 HTTP 上下文，便于单元测试。
3. **异步任务模式：**对于抓取、AI 增强等耗时操作，API 将任务描述写入 Redis 队列后即刻返回，Celery Worker 异步执行并写回结果。任务状态和失败重试由 Worker 管理，不阻塞核心 API 路径。

2.2 系统分层

前端和后端分别按层次组织：

前端分层（自顶向下，下层为上层提供服务）：

1. **视图层：**页面级组件（views），每个视图对应一个路由入口，负责页面组装和状态消费。
2. **组件层：**可复用 UI 组件（components），包含通用布局（AppShell、NavHeader）和业务组件（CaptchaField），消费 store 中的状态。
3. **状态管理层：**Pinia store（stores/），管理跨页面的认证状态和共享数据，调用 API 层获取数据。
4. **API 封装层：**Axios 请求封装（api/），负责 HTTP 调用、响应数据适配和字段重命名，被 store 和组件调用。
5. **工具层：**纯函数和组合式逻辑（utils/、composables/），放置排序、质量评分等可独立测试的业务逻辑，不依赖任何其他前端层。

后端分层（自顶向下）：

1. **接口层：**Flask 蓝图，负责 HTTP 路由、参数解析、JWT 校验和响应序列化。

- 2. **服务层**：封装业务规则和质量策略，包括内容解析、质量评分、去重、安全校验、审计和通知。
- 3. **持久层**：SQLAlchemy 模型，定义实体关系、约束条件、事务管理和数据序列化。
- 4. **基础设施层**：数据库（PostgreSQL/SQLite）、缓存与队列（Redis）、异步任务（Celery）、以及外部服务（邮件、搜索、AI）的客户端适配。

2.3 各层职责

层次	职责	代表实现
前端层		
视图层	页面组装、路由分发、状态消费	views、router、stores
组件层	可复用 UI 组件、通用布局	AppShell、NavHeader、PageState
状态管理层	跨页面状态管理、数据获取	Pinia stores（auth 等）
API 封装层	HTTP 请求、数据适配	api/ 模块
工具层	纯函数、组合式逻辑	utils/、composables/
后端层		
接口层	HTTP 路由、参数解析、JWT 校验	auth、activities 等蓝图
服务层	业务规则、质量、去重、审计、通知	services/*
持久层	实体定义、关系映射、约束、事务	Poster、User、Registration 等
基础设施层	数据库、缓存、队列、外部服务	extensions、config、Docker Compose

3 功能模块设计

3.1 模块划分

依据需求分析的功能域划分，系统包含以下七个核心功能模块和一个通用模块：

- 1. **活动发现模块**：活动列表、关键词搜索、分类/状态筛选、活动详情展示。
- 2. **身份认证模块**：图形验证码登录、邮箱注册、JWT 会话管理、角色鉴权。
- 3. **个人参与模块**：报名、取消报名、收藏、个人日历、ICS 导出。

- 4. 活动发布模块：草稿创建与编辑、提交审核、审核结果查看。
- 5. 审核管理模块：活动审核、批量操作、发布者申请审批、审核意见记录。
- 6. 知识与通知模块：知识节点关联、订阅管理、通知生成与已读标记。
- 7. 数据治理模块：数据源维护、抓取日志、内容质量分、疑似重复标识、审计日志。
- 8. 异常与通用模块：跨页面加载态、空态、错误态、401/403 统一处理。

3.2 模块职责

各模块在后端由独立的 Flask 蓝图承载,前端由对应的视图页面和 API 封装模块支撑:

模块	后端蓝图/服务	前端视图/组件
活动发现	posters 蓝图、search 蓝图	HomeView、SearchView、ActivityDetail
身份认证	auth 蓝图	LoginView、RegisterView、CaptchaField
个人参与	posters (报名/收藏端点)、calendar 蓝图	ProfileView、CalendarView
活动发布	posters 蓝图、poster_service	ActivityEditor、ActivityManagement
审核管理	review 蓝图、audit_service	ReviewView、AdminDashboard
知识与通知	knowledge 蓝 图、notification_service	KnowledgeManagement
数据治理	data_sources 蓝 图、quality_service、dedup_service	DataSourcesView
通用	全局异常处理、请求日志	PageState、AppShell

3.3 模块依赖关系

模块间的依赖遵循以下规则：

- 1. 身份认证模块被所有需登录的模块依赖，提供 JWT 校验和角色查询能力。
- 2. 活动发现模块无上游依赖，访客和未登录用户均可访问。
- 3. 活动发布模块依赖于身份认证模块（发布者角色校验），输出待审核活动供审核管理模块消费。
- 4. 审核管理模块在审核通过后触发知识与通知模块，生成关联关系和站内通知。

5. 数据治理模块作为管理后台能力，独立于用户侧流程，仅管理员可访问。
6. 通用模块被所有页面依赖，提供一致的异常处理、加载态和空态展示。

4 数据与接口设计

4.1 数据存储方案

系统采用混合存储策略：

1. **关系型数据库 (PostgreSQL / SQLite)**：存储所有业务实体，包括用户、活动、报名、收藏、日历事件、知识节点、数据源配置和审计日志。开发环境使用 SQLite 实现零配置启动，部署环境切换为 PostgreSQL。两者通过 SQLAlchemy ORM 抽象层无缝切换，业务代码无需感知底层数据库差异。
2. **Redis**：承载三方面职责——缓存（活动列表热点数据）、限流（登录和 API 频率控制）、消息队列（Celery 任务调度）。Redis 为可选依赖，不可用时核心业务仍可运行，仅限流和异步任务降级。
3. **文件存储**：用户头像和活动附件存储在本地文件系统，开发环境使用项目目录下的上传目录，生产环境建议切换为对象存储。

4.2 核心数据关系

系统核心实体关系如下：

1. **User—Activity (1:N)**：一个用户可创建多个活动，活动必须有创建者。
2. **User—Registration—Activity (M:N)**：用户与活动之间的报名关系，(user_id, activity_id) 唯一约束保证幂等。
3. **User—Favorite—Activity (M:N)**：收藏关系，同样由唯一约束保证幂等。
4. **User—CalendarEvent—Activity (M:N)**：个人日历事件，支持 ICS 导出。
5. **Activity—KnowledgeNode (M:N)**：通过中间表 PosterNode 关联，(poster_id, node_id, relation_type) 唯一。
6. **User—AuditLog (1:N)**：关键管理操作必须记录审计日志，包含操作者、动作、目标和时间。

4.3 模块接口

模块间以 RESTful API 为通信契约：

1. **接口风格**：所有接口遵循 REST 风格，使用 HTTP 方法语义（GET 查询、POST 创建、PUT 更新、DELETE 删除），统一返回 JSON 格式。

- 2. **认证方式:** 接口通过 JWT Bearer Token 认证, Token 在登录时签发, 前端通过 Axios 拦截器自动附加到每个请求的 Authorization 头。
- 3. **统一响应结构:** 非分页的成功响应直接返回数据体(如 {"id": 1, "title": "..."}); 分页的成功响应返回 {items: [...], page, per_page, total}, 默认每页 20 条。错误响应统一为 {"error": " 描述信息"} 格式; 多字段校验错误返回 {"error": " 描述", "field_errors": {"title": " 标题不能为空"}}。HTTP 状态码与语义一致(400 参数错误、401 未认证、403 越权、404 资源不存在、500 服务端错误)。
- 4. **时间格式:** 所有时间字段使用 ISO 8601 格式 (YYYY-MM-DDTHH:mm:ss), 前后端统一解析。

4.4 外部接口

系统与三类外部服务交互, 均为可选依赖:

- 1. **邮件服务:** 用于发送验证码和通知邮件。通过环境变量配置 SMTP 参数; 服务不可达时注册流程降级为提示用户稍后重试, 不影响已登录用户的操作。
- 2. **搜索服务:** 提供全文检索能力。内部搜索基于数据库 LIKE 查询为默认实现(注意: 前导通配符 %keyword% 无法利用数据库索引, 千级数据量下性能可接受, 万级以上建议启用外部搜索引擎); 外部搜索引擎(如 Elasticsearch)为可选增强; 外部搜索超时 10s, 超时后回退到内部搜索。
- 3. **AI 服务:** 用于活动摘要生成和内容推荐。通过 REST API 调用 LLM 服务, 超时 30s, 超时或不可用时返回基础摘要或默认推荐列表, 不影响活动浏览和审核核心路径。

5 关键设计决策

5.1 技术选型

决策	选型方案	理由	替代方案
前端框架	Vue 3 + Type-Script	类型安全、组合式 API 逻辑复用、社区活跃	React、Angular
状态管理	Pinia	官方推荐、Type-Script 友好、API 简洁	Vuex、reactive composables

决策	选型方案	理由	替代方案
后端框架	Flask + 蓝图	轻量灵活、生态成熟、学习成本低	Django、FastAPI
ORM	SQLAlchemy	支持多数据库切换、迁移工具链成熟	Peewee、Django ORM
认证	JWT + RBAC	无状态、跨域友好、角色边界明确	Session/Cookie、OAuth
容器化	Docker + Compose	环境可复现、单命令部署	Kubernetes、手动部署

5.2 关键方案说明

- 1. **RBAC 权限模型：**系统定义三种角色——viewer（默认）、publisher（发布者）、admin（管理员）。后端在每个受限接口通过装饰器校验 JWT 中的角色声明，前端路由守卫仅做导航引导，不替代服务端授权。
- 2. **活动审核状态机：**活动状态转换由服务层集中控制，包含四种状态——draft（草稿）→ pending_review（待审）→ published（已发布）| rejected（已驳回），驳回后可重返 draft 修改。非法状态转换返回 400 错误。
- 3. **领域对象适配：**前端以 Activity 为领域概念，后端持久化模型中为 Poster。前端 API 层负责字段重命名和格式转换，页面组件不直接引用后端模型字段，隔离数据库重构风险。

5.3 方案权衡

序号	权衡点	所选方案的收益	付出的代价
1	前后端分离 vs 一体式开发	解耦迭代、Mock 先行、独立部署	接口协调成本增加，需契约矩阵和联调清单管理
2	JWT 无状态 vs Session 有状态	无需服务端存储会话、跨域友好	Token 吊销需黑名单机制，生产需配合短生命周期和刷新 Token
3	关系型 vs NoSQL	事务一致性、复杂查询支持、唯一约束兜底幂等	Schema 变更需迁移工具，不适合超高并发写入
4	Docker Compose vs 裸机部署	环境一致、依赖可复现、学习成本低	非高可用方案，生产需演化到编排平台（K8s）和托管数据库

6 架构质量属性

6.1 可维护性

- 1. **分层隔离**：前端按 views / components / stores / api / composables+utils 分层，后端按接口 / 服务 / 持久层分层，每层职责明确，下层为上层提供服务，修改范围可控。
- 2. **领域名称适配**：前端 API 层统一将后端的 Poster 适配为前端的 Activity，页面代码不直接依赖后端持久化字段。后端表结构重构时，仅需修改 API 适配层，前端组件不受影响。
- 3. **测试覆盖**：服务层函数为纯业务逻辑，不依赖 HTTP 上下文，可独立编写单元测试。后端服务层和前端业务工具函数测试覆盖率目标不低于 70%，核心模块（审核、权限、质量）不低于 85%。
- 4. **配置管理**：所有环境差异项通过 .env 文件管理，仓库只保留 .env.example 模板。密钥、数据库连接串和外部服务地址均不进入版本控制。

6.2 可扩展性

- 1. **模块化扩展**：后端使用 Flask 蓝图组织路由，新增功能域只需注册新蓝图并声明路由前缀，不影响已有模块。前端对应新增视图页面和 API 封装模块即可。
- 2. **外部能力可插拔**：邮件、搜索、AI 等服务均通过环境变量开关控制启用状态，服务不可达时系统以降级模式运行。新增外部服务只需实现适配接口并注册到服务层。

3. **数据源扩展：**数据源抓取采用插件化设计，新增抓取目标只需配置 URL、允许域和抓取模式，无需修改抓取引擎核心逻辑。

6.3 性能与可靠性

1. **异步处理：**Celery Worker 承担抓取、AI 处理、定时通知等耗时任务，API 服务通过 Redis 队列异步派发，避免 Web 请求阻塞。Worker 崩溃不影响 API 可用性，任务在 Redis 中持久化等待重新调度。
2. **超时与降级：**外部调用设置超时阈值——搜索 10s、AI 服务 30s。超时后返回降级结果（基础搜索或默认推荐），核心活动浏览、认证和审核流程不因外部超时而阻塞。
3. **缓存策略：**热点活动列表和可枚举字典数据通过 Redis 缓存，减少数据库查询频率。缓存过期时间为 5-30 分钟，视数据更新频率调整。
4. **安全防护：**认证端点实施验证码和限流防止暴力破解；密码以哈希形式存储；外部 URL 抓取校验协议、域名和回环地址以降低 SSRF 风险；所有审核和管理操作记录审计日志以供追溯。
5. **数据可靠性：**报名、收藏等关系操作依赖数据库唯一约束提供最终幂等保证。写操作遵循“校验—领域处理—事务提交”的顺序，单次请求内的状态更新、审计记录和通知触发在同一事务边界内完成。